

Training a Pokémon Doubles Battle Agent with Deep Reinforcement Learning

CPTS 440 — Artificial Intelligence
Final Project Report

May 2026

Abstract

This report presents the design, formulation, and empirical evaluation of a deep reinforcement learning agent trained to play Pokémon VGC-style Doubles battles. The problem is framed as a cooperative two-agent partially observable Markov decision process (POMDP), solved using Proximal Policy Optimization (PPO) with a shared actor-critic neural network. A custom Gymnasium-compatible environment wraps a locally running Pokémon Showdown game server, providing a training signal through shaped rewards. The agent is first bootstrapped against a random opponent and then refined through self-play, creating an implicit curriculum where opponent difficulty scales with agent capability. Empirical results demonstrate progressive improvement in win rate across both training regimes, validating the approach for large combinatorial, partially observable, turn-based strategy games.

1. Introduction

Competitive Pokémon is a turn-based strategy game that, on its surface, may appear to be a simple card game. In practice, however, it constitutes one of the most complex discrete decision-making environments that has been seriously analyzed in the game-AI literature. The standard competitive format is Doubles, where two Pokémon per side are active simultaneously, and both players commit their moves at the same time each turn without seeing the opponent's selection first. This simultaneous commitment structure, combined with a partially hidden opponent team and a combinatorially large joint action space, places the problem firmly in the category of imperfect information extensive form games.

The motivation for this project is threefold. First, Pokémon Doubles provides a rich testbed for evaluating reinforcement learning (RL) methods under conditions that are common in

real-world applications but underrepresented in canonical RL benchmarks: partial observability, joint discrete actions, long-horizon credit assignment, and a large but structured action space with many illegal joint combinations. Second, the existence of Pokémon Showdown—a high-quality, open-source battle simulator with a well-maintained Python interface—makes the environment both reproducible and extensible. Third, the project serves as a case study in *how* to apply the full deep-RL pipeline—environment design, feature engineering, network architecture, and training curriculum—to a domain that was not built with machine learning in mind.

The remainder of this report is organized as follows. Section 2 formalizes the problem as an MDP. Section 3 describes the software environment stack. Section 4 details the observation and action representations. Section 5 describes the training curriculum and self-play strategy. Section 6 presents empirical results. Section 7 discusses limitations and future directions.

2. Problem Formulation

2.1 Game Description

A Pokémon Doubles battle proceeds as follows. Each player brings a team of six Pokémon but only two are on the field at any time. On each turn, both players simultaneously choose an action for each of their active Pokémon. An action is either:

- **Use a move:** up to four moves are available per Pokémon, each possibly targeting either opponent or ally.
- **Switch:** replace the active Pokémon with one of up to four benched, non-fainted teammates.
- **Terastallize and use a move:** a one per battle transformation that changes a Pokémon’s type (only one player Pokémon may Terastallize per battle).

The battle ends when all six Pokémon on one side have fainted.

2.2 Reward Function

A sparse win/loss signal at the end of each battle is insufficient for efficient learning given the long horizon of a typical battle (20–50 turns). We therefore use a *shaped* per-turn reward provided by poke-env’s reward computing helper:

$$r_t = 100 \cdot \Delta f_{\text{opp}} - 100 \cdot \Delta f_{\text{self}} + 100 \cdot \Delta \text{HP}_{\text{opp}} - 100 \cdot \Delta \text{HP}_{\text{self}} + 20 \cdot \Delta S_{\text{opp}} + 10 \cdot \mathbf{1}[\text{victory}] \quad (1)$$

where Δf denotes the change in fainted Pokémon count, ΔHP denotes the fractional HP change, and Δs denotes newly inflicted status conditions. The relative magnitudes encode domain knowledge: fainting a Pokémon (a permanent resource loss) is weighted more heavily than HP damage, which is itself weighted more heavily than status conditions.

3. Environment Architecture

3.1 Software Stack

The environment is implemented as a three-layer stack:

1. **Pokémon Showdown** — An open-source battle simulator written in TypeScript/Node.js, run locally with the `--no-security` flag to disable authentication. It handles all game logic, legality enforcement, RNG, and damage calculation.
2. **poke-env** — A Python library that communicates with the Showdown server over a WebSocket connection. It parses the streaming protocol messages into structured Python objects (Double Battle, Pokemon, Move, etc.) and exposes a Gymnasium compatible `DoublesEnv` base class.
3. **CustomEnv** — Our extension of `DoublesEnv` that overrides the observation embedding, action masking, and reward calculation to produce a training-ready environment.

3.2 Custom Environment Design

`CustomEnv` inherits from `DoublesEnv` and provides four critical overrides:

embed battle() constructs the fixed-length observation vector from the current Double Battle object. It normalizes all features to be approximately in $[-1,1]$, handles missing/unknown values with sentinel encodings, and maintains incrementally learned dictionary mappings for nominal features such as items and abilities.

get mask() computes the binary validity mask over the 11,449-dimensional joint action space. Valid individual sub-actions are found via `valid_orders()`, converted to indices, and then combined via outer product. Illegal combinations (both passing, both switching to the same slot, both Terastallizing) are then zeroed out.

Other: It is worth highlighting the other functions that have been overwritten in `CustomEnv`. Most of these rewritten functions increase the speed of the program. The base `poke-env` code used many inefficient calls to the server, which increased training time to upwards of 30 seconds for a single game. In a recent commit that can be found in the Github, this was optimized down to around .26 seconds per game. That is a speedup of over 100 times.

4. State and Action Representation

4.1 Observation Vector

The observation vector is a flat float32 array of fixed length, constructed by concatenating the following components:

Move features (8 values). For each of the (up to four) available moves of each active Pokémon:

- **Base power:** normalized by dividing by 100.
- **Type effectiveness multiplier:** the damage multiplier of the move's type against the active opponent Pokémon (0.25×, 0.5×, 1×, 2×, or 4×).

Team health summary (2 values). The fraction of fainted Pokémon on each team (values in [0,1]).

Per Pokémon embedding (28 features × 12 Pokémon). All six Pokémon on the player's team and all (known) opponent Pokémon are individually embedded. Each embedding contains:

- Active status (binary)
- Current HP fraction
- Held item (dictionary index, normalized)
- Type 1 and Type 2 (enum values, normalized)
- Six base stats (HP, Atk, Def, SpA, SpD, Spe), each divided by 255
- Status condition (normalized enum value)
- Six stat stage boosts (each divided by 8; range [-1,1])
- Level (divided by 100)
- Six actual in-battle stats (normalized)
- Max and current HP (each divided by a species max of 714)
- Ability (dictionary index, normalized)

Unknown or unrevealed opponent Pokémon are encoded with -1 sentinel values to signal to the network that the information is unavailable.

Item and ability mappings. Items and abilities are nominal categorical variables with hundreds of possible values. They are mapped to integer indices via a dictionary that is learned incrementally during training: the first time a new item or ability is encountered, it receives the next available index. These dictionaries persisted to disk so they survive training restarts. Values are normalized by dividing by the current dictionary size.

5. Training Curriculum

5.1 Two-Phase Training

Training proceeds in two phases that correspond to two operational modes in the implementation.

Phase 1 — Bootstrapping against a Random Opponent (Mode 0). In the first phase, the learning agent (P1) faces a `RandomPlayer` that samples uniformly from its valid actions at each turn. This provides an easy initial training signal: the agent quickly learns basic type-matching, to not waste moves, and to avoid self-damaging choices. Only P1’s observations, actions, and gradients are tracked; P2 acts independently and contributes no learning signal.

Phase 2 — Self-Play (Mode 1). In the second phase, a copy of the current best checkpoint is loaded as P2. P2 runs in `eval()` mode (frozen weights, no gradient tracking), acting as a stationary but increasingly strong opponent. P1 continues training, and the best checkpoint is periodically updated with the current P1 weights.

Self-play creates an implicit curriculum: as P1 improves, the opponent it must overcome at the next checkpoint update becomes stronger as well. This prevents the agent from overfitting strategies that only work against weak or random opponents.

5.2 Data Collection Overview

The simulator begins with effectively random decision making (unless booting from a checkpoint). It plays through a game while making decisions based on observations (actor), then saves each pair of observations and decisions to storage at the end of each turn. This is then used to backpropagate the network using the reward function and the critic, an identical neural network that predicts the rewards. Using PPO for clipping values to introduce variety in decision making, the algorithm then goes into the next episode having (ideally) learned from the past states.

5.3 Episode Structure

Each training episode consists of a complete battle (up to max cycles = 125 steps) against the current opponent. At the end of each episode:

1. The collected trajectory (observations, actions, rewards, log-probabilities, values, termination flags) is stored in a replay buffer of shape $[125, n_{\text{agents}}, n_{\text{obs}}]$.
2. A single PPO update pass is run over mini-batches drawn from the episode buffer.
3. The win/loss result is appended to sliding windows of size 10 and size 50 for live win-rate monitoring.
4. The model checkpoint is saved to disk as agent.pt.

5.4 Post-Training Validation

After training, the saved agent is loaded into an AlgoPlayer wrapper, which implements the poke-env Player interface. This allows the agent to be evaluated against any other Player-derived opponent (random, rule-based, or another neural-net agent) by running a series of asynchronous battles via asyncio. Win rates are computed over 1000 validation games and reported in a cross-evaluation table.

6. Results

This section reports empirical results from training experiments conducted on the fixed doubles format (gen9doublesou). Checkpoint versions agent_v1.pt, agent_v1.1.pt, and agent_last.pt represent successive stages of training. All results were sampled on the latest version of the program and may not reflect newer or past weights.

6.1 Training Win Rate

Table 2: Training win rates at key checkpoints (Mode 0 — vs. Random Opponent).

Checkpoint	Iterations	Overall WR(%)	Last-50WR(%)
agent-v1.pt	4096	76.3	84
agent-v1.1.pt	2048	88.4	92
agent-last.pt	1024	90.1	98

6.2 Self-Play Training

Table 3: Self-play (Mode 1) win rates vs. frozen prior checkpoint.

Episodes	Overall WR (%)	Last-50 WR (%)
100	32.3	38
250	51.6	64
500	64.3	68
1024	73.5	76

7. Discussion

7.1 What Worked

Action masking proved essential. Without it, the policy wastes significant capacity modeling illegal action combinations. With masking applied post-logit and pre-sample, the agent converges faster and avoids degenerate behaviors such as attempting to switch to an already active Pokémon.

The two-phase curriculum (random opponent first, self-play second) was likewise important. Starting directly with self-play from a random initialization results in two agents that are both equally bad, producing a noisy and uninformative training signal. The random opponent bootstrap phase provides a stable lower bound from which self-play can begin to create meaningful competitive pressure.

The shaped reward function significantly accelerated early learning. A sparse win/loss reward alone gives the agent no feedback for the first 20–50 turns of a battle. Rewarding incremental HP damage and status infliction provides dense feedback that guides the policy toward aggressive, offensive play before it has learned to plan at the strategic level.

The model demonstrated some level of strategy. While simplistic, visual simulations show that the model tends to favor high-power attacking moves, as well as spread moves that can damage multiple Pokémon at once. This is likely due to the reward function, which favors overall damage, but shows that the model can learn given the right parameters and situations.

7.2 Limitations

Several simplifications remain in the current implementation:

Incomplete feature embedding. The observation vector does not currently include weather conditions, field effects (entry hazards, screens), side conditions, or the Terastallize type of an opponent Pokémon. These features could meaningfully affect win rate at higher levels of play.

Training time. While learning was demonstrated in fixed teams and RNG, self-play found limited results after some training sessions. We attribute this to needing time learning and tuning hyperparameters and network structure, as self-play was only able to be completed within the last month.

Random self-play. The best results occurred on fixed team lists, where we could see evidence of learning and strategy. In fully random battles, with random Pokémon, abilities, and items, we saw very slow learning. Some models found slight results, but most performed around a low 40-60% baseline with high loss. With more time for refinement, we would have hoped to find learning beyond our structured team sets.

7.3 Future Work

Several natural extensions follow from this work:

- **Richer feature engineering** incorporates weather, terrain, side conditions, and move secondary effects.
- **Factored action head:** replace the flat joint-action output with two independent heads of size 107, one per active Pokémon slot.
- **Population-based self-play:** maintain a league of checkpoints at varying skill levels; the current agent is matched against randomly sampled historical opponents, preventing cycling.
- **Opponent modeling:** incorporates an explicit representation of inferred opponent strategy, enabling the agent to exploit predictable opponents.
- **Team building agent:** extend the system to choose teams as well as in-battle actions, turning the full competitive cycle into a learning problem.

8. Conclusion

This project demonstrated that Proximal Policy Optimization with action masking and a self-play curriculum can train a competent Pokémon Doubles agent from scratch, using only game outcomes and shaped intermediate rewards as a training signal.

The key technical contributions are:

1. A complete Gymnasium-compatible environment wrapping a live Pokémon Showdown server, enabling reproducible RL training.
2. A joint-action encoding scheme that maps the 107×107 Doubles action space to a flat integer representation compatible with standard categorical policy heads, with combinatorial action masking to enforce legality.
3. An incremental feature-engineering pipeline that handles the nominal categorical variables (items, abilities) present in Pokémon without requiring fixed, pre-specified vocabulary.
4. A two-phase training curriculum that bootstraps on a random opponent and then refines via self-play, providing a stable and scalable training signal.

The empirical results confirm that the agent achieves win rates substantially above random play and continues to improve under self-play, validating the overall approach. The Pokémon Doubles domain offers a productive test for future work on multi-agent RL, imperfect information, and large combinatorial action spaces.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [2] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. Araújo, “CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms,” *Journal of Machine Learning Research*, vol. 23, no. 274, pp. 1–18, 2022.
- [3] G. Delmas, “poke-env: A Python Interface for Creating Pokémon Battling Agents,” 2020. <https://github.com/hsahovic/poke-env>
- [4] Pokémon Showdown, “An Open-Source Pokémon Battle Simulator,” <https://github.com/smogon/pokemon-showdown>

- [5] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in *Proceedings of the 32nd International Conference on Machine Learning*, 2015.
- [6] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. Cola, T. Deleu, M. Goel, K. Lahlou, A. Suri, G. Thakkar, A. Shinde, H. Huang, D. Balestrieri, M. Moulin-Frier, and D. Bhatt, "Gymnasium," *Zenodo*, 2023. <https://doi.org/10.5281/zenodo.8127026>